

CloudX 集成指南：在 AdMob 中使用虚拟探针自定义事件作为价格代理

版本 3.3（2026 年 6 月 23 日）

概述

本指南面向 AdMob 发布商，介绍如何将 CloudX 作为并行竞价方（parallel bidder）与 AdMob 聚合（mediation）集成，并使用虚拟自定义事件探针作为价格代理（price proxy）。

本指南聚焦于 **AdMob 端配置以及价格代理读取器**。CloudX SDK 自身的接入（并行竞价逻辑、出价比较流程、渲染决策）由您的 CloudX 集成对接人并行处理，属于另一独立部分。

通过本指南，您将获得：

1. AdMob 平台的精确配置（一个聚合组（MG）、真实竞价方 or AdMob Network、自定义事件探针）。
2. **Android（Kotlin）** 和 **iOS（Swift）** 虚拟探针范例源码。
3. 集成期间用于应用埋点的**日志记录模板**。
4. **部署注意事项**——来自 CloudX × AdMob 生产部署的运营经验。

为什么需要这样做

在标准的并行竞价（parallel bidding）架构中，您的应用同时调用 CloudX 和 AdMob，比较两者返回的价格，并渲染较高出价的广告。当两个需求来源都提供单次展示级别（impression-level）的收益数据时，整个流程很简单。

然而，AdMob **不会**向大多数发布商提供单次展示收益数据（ILRD，Impression-Level Revenue Data）API。如果没有 ILRD API，您的应用就无法实时直接比较 AdMob 的实际 eCPM 与 CloudX 的拍卖出价。

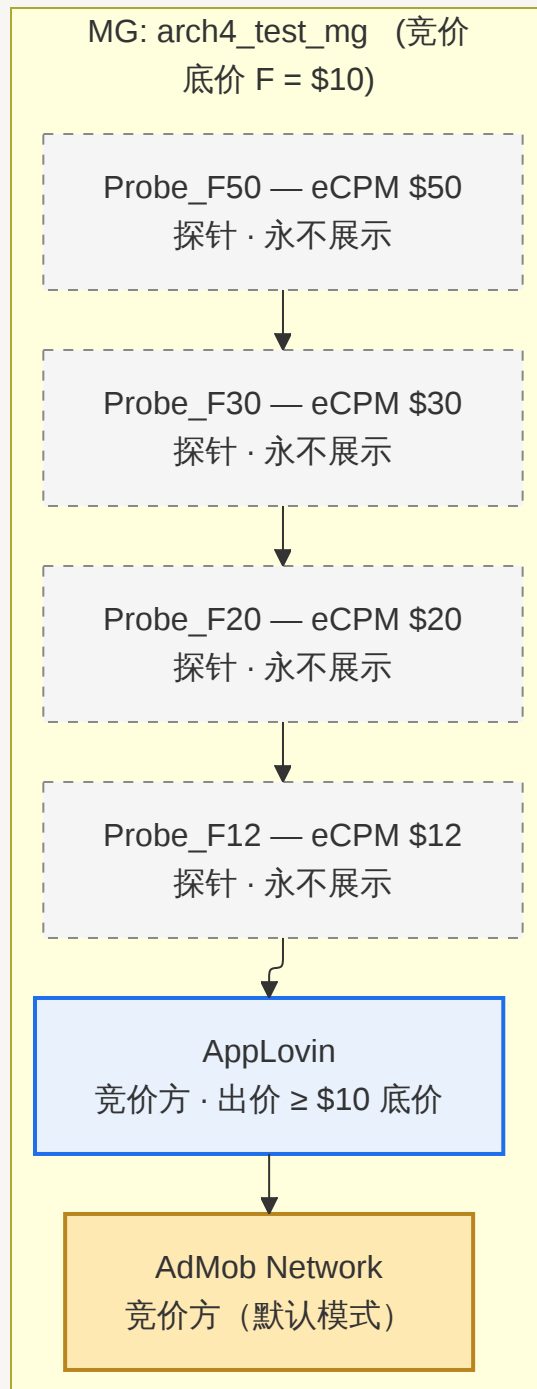
本指南的方案是：在单一聚合组内，于已知的层级价位上放置**虚拟自定义事件探针（dummy custom-event probes）**。这些探针永远不会真正展示广告——它们触发后立即返回无填充（no-fill）——但它们的标签（Label）会通过 `ResponseInfo` 回传，从而显示 AdMob 瀑布流（waterfall）在最终胜出方出现之前到达了哪些层级。最低被触发的探针即为 AdMob 实际填充价格的紧上界（tight upper bound），您的应用据此与 CloudX 出价进行比较。该机制适用于所有发布商。

进阶（AdMob Pro / UVS or KVP）：已使用用户价值分层（User Value Segmentation）或键值对（Key Value Pairs）的发布商，还可在分层聚合组内叠加探针，以获得额外的子分层粒度。

工作原理

1. 您的应用并行调用 **CloudX SDK** 与 **AdMob SDK**。CloudX 直接返回其出价。
2. AdMob 按 eCPM 降序遍历该聚合组（MG）的拍卖，向实时竞价合作伙伴（AppLovin、ironSource、AdMob Network 等）请求出价，并依次尝试每一条瀑布流广告位。
3. **eCPM 较高的探针先触发**——其适配器被调用、立即返回无填充，该次尝试会被记录到 `responseInfo.adapterResponses[*]` 中。
4. 最终由真实竞价方、AdMob Network 或某个较低 eCPM 的探针展示该次广告。
5. AdMob 返回 `ResponseInfo`。客户端遍历 `adapterResponses[*].adSourceInstanceName`，找出所有匹配 `Probe_F` 的标签，取**最低**已触发探针的 eCPM——这就是胜出方价格的最紧上界（瀑布流按 eCPM 降序遍历，因此每个已触发探针都位于最终胜出方之上）。
6. 应用计算 `proxyPrice = lowest_fired_probe`，并与 CloudX 的出价进行比较。如果 CloudX 出价超过 `proxyPrice`，则 CloudX 胜出；否则由 AdMob 展示。**提升探针密度能有效提升价格精度**：最低已触发探针即为上界，因此越靠近 AdMob 实际填充价位的探针越密集 = 上界越紧 = CloudX 超额支付越少（参见步骤 1 第 5 项）。当没有探针触发时（胜出方位于所有探针之上），`proxyPrice` 为 null——默认由 AdMob 展示（参见步骤 3）。

聚合组（MG）结构：



单一聚合组、单一广告单元。该图展示**运行时**按 eCPM 排序的视图：探针（位于 AdMob 瀑布流（*Waterfall*）区域、采用手动 eCPM 的自定义事件）与竞价方（位于竞价（*Bidding*）区域的 AppLovin 与 AdMob Network）交错排列。探针仅传递价格

信号；真正展示广告的是 AppLovin 和 AdMob Network。每个探针的 eCPM 都高于 \$10 的竞价底价——低于底价 F 的探针永远不会触发。上图所用的阶梯（\$12 / \$20 / \$30 / \$50）为了示意清晰相对稀疏；在生产环境中，密度调优（步骤 1 第 5 项）才是让 proxyPrice 贴近 AdMob 实际价格的杠杆。

典型触发模式（一次示例展示，AppLovin 中段出价 \$25）：



已触发探针集合 = {30, 50}（可在 responseInfo.adapterResponses 中查看）。AppLovin 越过较低层级的探针并展示广告；AdMob Network 与位于其下方的探针被截断、不会被记录。最低已触发探针 = \$30 → proxyPrice = \$30。CloudX 的出价与 \$30 进行比较：若 CloudX > \$30，则 CloudX 胜出（明显高于 AdMob 实际填充价，该价位介于 \$20 与 \$30 之间）；否则该次展示仍归 AdMob。

步骤 1 — AdMob 平台配置

以下是 AdMob 执行的点击路径。每一步都会标注该项配置最终会填充到 ResponseInfo 的哪个字段。

1. **创建广告单元 (Ad unit)**。Apps → 您的测试应用 → Ad units → Add ad unit。命名为 arch4_test_interstitial，格式为插屏 (Interstitial)。仅一个广告单元。
2. **创建一个聚合组 (Mediation group)**。Mediate → Create mediation group。状态为活跃 (Active)，格式为插屏，定向到刚才创建的广告单元。聚合组名称会填充至 responseExtras["mediation_group_name"]（适合作为遥测数据的合法性校验通道）。
3. **设置 MG 竞价底价 (F) (可选)**。在 MG 内，您可以设置 Optimize → Floor eCPM（如 \$10）以过滤掉低于底价的出价——这是 AdMob 的常规设置，与代理机制本身无关。F 不会出现在 proxyPrice = lowest_fired_probe 的公式中。它对代理机制的唯一影响是约束探针放置：任何不高于 F 的探针都不会触发（竞价方会先胜出），因此探针必须严格高于 F（见下文第 5 项）。请将广告单元级别的 eCPM 底价保持为 AdMob 默认值；此处的 F 仅指 MG 级别的值。
4. **添加自定义事件探针 (在全局定义自定义事件)**。Mediate → Custom events → Add custom event。为每个探针层级配置：
 - 各层级表单字段：

- **Label（标签）**——这是关键字段。它会填充 `ResponseInfo` 中的 `adSourceInstanceName`，是您读取器中正则表达式所依赖的通道。请使用一种能编码层级 eCPM 的可预测格式，例如 `Probe_F12`、`Probe_F20`、`Probe_F30`、`Probe_F50`。最长 255 字符。
- **Manual eCPM（手动 eCPM，美元）**——决定瀑布流排序。请与 Label 中的 eCPM 保持一致：`12`、`20`、`30`、`50`。AdMob 用户界面提示语为 "The manual eCPM determines the order of the ad source when serving ads"，这正是代理机制所需要的。

Add custom event

Label ⓘ

9 / 255

Manual eCPM (\$ USD) ⓘ

ⓘ The manual eCPM determines the **order of the ad source** when serving ads. This value will also be used for analysis and reporting purposes. Ensure this value is accurate as it will be used for **eCPM and estimated earnings in reports**

Cancel
Continue

- 阶梯设计原则：
 - 始终包含一个足够高的顶层探针，以保证每次都至少有一个探针触发。可防止 `proxyPrice` 出现 null。
 - 采用更密集的探针间距以获得更紧（更精确）的上界。更多探针（尤其是在 AdMob 实际填充集中的价位带）可减少 CloudX 的超额支付。
 - 所有探针都必须严格高于 MG 底价 `F`。≤ `F` 的探针永远不会触发。
5. 将每个自定义事件映射到广告单元（按广告单元映射）。保存自定义事件后，AdMob 会立即提示添加映射；之后也可在该自定义事件的行 → *Add mapping* 进入。从第 1 项中选择您的测试广告单元，并填写：
- **Mapping name（映射名称）**——在该广告单元的映射列表中必须唯一。最简单的约定是直接复用 Label 字符串（如 `Probe_F20`）。**该名称不会出现在**

ResponseInfo 中——它仅是映射行的用户界面级标识符。

- **Class Name (类名)** ——表单要求字段。请使用完全限定的 Android 类名，以及匹配的 iOS Objective-C 名称，使该广告位在平台中表现为一个普通的自定义事件。例如推荐：Android 用 `com.example.mediation.FloorSignalAdapter`，iOS 用 `FloorSignalAdapter`（与步骤 2 中的 `@objc(...)` 名称一致）。是否真正发布该适配器类由您决定（步骤 2 是可选的）；当 classpath 上没有该类时，该广告位仍会在 `adapterResponses` 中显示其 Label 并返回无填充——这正是代理机制所需要的。
- **Parameter** ——可选的替代编码方式。Label（例如 Probe_F12）已经可以完整地返回层级 eCPM，因此此字段并非必需。如果您希望将底价作为类型化的整数提取，而不是使用正则表达式解析 Label，可使用 JSON 字符串如 `{"floor": 12}`，并从每个 `adapterResponses[*]` 空填充条目中读取 `credentials["parameter"]`；有关示例，请参阅 步骤 3 — 使用 Parameter 字段的替代读取器。如果留空，AdMob 会将 `parameter` 从 `credentials` 中省略，并且 Label 路径仍然是标准路径。您可以选择适合您读取器的任何通道；混合使用每行项目的方式是可以的。
- **Network eCPM (可选, "Reference only")** ——请留空。表单将其标记为 "Reference only"；它不会影响运行时的排序或筛选。运行时唯一起作用的 eCPM，是您在 Stage 1 中设置的 Manual eCPM。

Mapping name Probe_F5	\$Network eCPM (optional) Enter eCPM floor from the network ad unit. (Reference only)
Class Name com.example.mediation.FloorSignalAdapter	Parameter (optional)

6. **添加 AdMob Network。** 默认情况下，AdMob Network 位于 *Bidding*（竞价）区域（平台会自动放置）。如果您在权衡哪种模式更适合您的设置，请与 CloudX 客户经理（AM）讨论。最终的 MG 结构如下：

MG: arch4_test_mg （竞价 eCPM 底价：\$10）
竞价（Bidding）区域（针对 \$10 底价的实时出价）：

```

AppLovin (或 ironSource / Pangle)
AdMob Network          # 默认模式；与其他参与方一同竞价
瀑布流 (Waterfall) 区域 (来自 Stage 1 的 Manual eCPM，按降序
遍历)：
    Custom Event, Label: Probe_F50, Manual eCPM: $50 →
Mapping(testy_int): { name=F50, class=com.example.mediatio
n.FloorSignalAdapter, param={"floor":50} }
    Custom Event, Label: Probe_F30, Manual eCPM: $30 →
Mapping(testy_int): { name=F30, class=com.example.mediatio
n.FloorSignalAdapter, param=空 }
    Custom Event, Label: Probe_F20, Manual eCPM: $20 →
Mapping(testy_int): { name=F20, class=com.example.mediatio
n.FloorSignalAdapter, param=空 }
    Custom Event, Label: Probe_F12, Manual eCPM: $12 →
Mapping(testy_int): { name=F12, class=com.example.mediatio
n.FloorSignalAdapter, param={"floor":12} }

```

上例展示的是 **Android** 映射行——平台的 "Class Name" 使用完全限定的 Android 类名。在 iOS 端，AdMob 的 iOS 自定义事件平台使用无前缀的 Objective-C 名称 `FloorSignalAdapter`（与步骤 2 中的 `@objc(...)` 属性一致）。

步骤 2 — 客户端读取模式

读取 `responseInfo.adapterResponses` (Android) / `responseInfo.adNetworkInfoArray` (iOS)，找出每一个已触发的 `Probe_F` * 标签，取**最低值**作为 `proxyPrice` ——即 AdMob 填充价格的上界。最低已触发探针之所以是最紧上界，是因为瀑布流按 eCPM 降序遍历：每个已触发探针都位于最终胜出方之上。当没有探针触发时（罕见，但当所有探针层级都低于胜出方 eCPM 时可能发生），函数返回 `proxyPrice = null`，表示"没有可用的上界"。

当 `proxyPrice == null` 时，默认由 **AdMob 展示**。null 意味着 AdMob 的填充价格高于所有探针，因此没有可与 CloudX 进行比较的上界；将该次展示让给 AdMob 是保守的做法，可保护既有的 AdMob 收益。如果您的监控显示 null 比例超过填充总量的约 1%，请提高**顶层探针**——请基于您历史上的 AdMob 收益分布来调整顶层探针的高度。

Android (Kotlin) Example

```

import com.google.android.gms.ads.interstitial.InterstitialAd

private val PROBE_REGEX = Regex("""Probe_F(\d+)""")

data class FloorProxyResult(
    val firedProbeFloors: List<Int>,
    val lowestFired: Int?,
    val proxyPrice: Int?,
    val winnerSourceName: String?,
    val winnerInstanceName: String?,
    val mgName: String?,
)

fun computeFloorProxy(ad: InterstitialAd): FloorProxyResult {
    val resp = ad.responseInfo

    val firedProbeFloors = resp.adapterResponses
        .mapNotNull { it.adSourceInstanceName }
        .mapNotNull { PROBE_REGEX.find(it)?.groupValues?.get
(1)?.toIntOrNull() }
        .sorted()

    // 最低已触发探针 = AdMob 价格的最紧上界（瀑布流按 eCPM 降序遍
    历）。
    // 当没有探针触发时为 null — 胜出方位于所有探针之上；没有可用上
    界。
    val lowestFired = firedProbeFloors.minOrNull()
    val proxyPrice = lowestFired

    return FloorProxyResult(
        firedProbeFloors = firedProbeFloors,
        lowestFired = lowestFired,
        proxyPrice = proxyPrice,
        winnerSourceName = resp.loadedAdapterResponseInfo?.ad
        SourceName,
    )
}

```

```

        winnerInstanceName = resp.loadedAdapterResponseInfo?.
adSourceInstanceName,
        mgName = resp.responseExtras.getString("mediation_gro
up_name"),
    )
}

```

接入到您的 `InterstitialAdLoadCallback` :

```

import com.google.android.gms.ads.interstitial.InterstitialAd
LoadCallback
import com.google.android.gms.ads.LoadAdError

class TestAdLoadCallback : InterstitialAdLoadCallback() {
    override fun onAdLoaded(ad: InterstitialAd) {
        val result = computeFloorProxy(ad)
        android.util.Log.i("FloorProxy", result.toString())
        // 生产环境: 若 result.proxyPrice == null, 则默认由 AdMob
展示。
        // 否则, 将 result.proxyPrice 与 CloudX 的并行出价进行比
较。
    }

    override fun onAdFailedToLoad(error: LoadAdError) {
        android.util.Log.w("FloorProxy", "no fill: $error")
    }
}

```

iOS (Swift) Example

```

import GoogleMobileAds

private let probeRegex = try! NSRegularExpression(pattern:
#"Probe_F(\d+)"#)

```

```

struct FloorProxyResult {
    let firedProbeFloors: [Int]
    let lowestFired: Int?
    let proxyPrice: Int?
    let winnerSourceName: String?
    let winnerInstanceName: String?
    let mgName: String?
}

func computeFloorProxy(ad: InterstitialAd) -> FloorProxyResult? {
    guard let resp = ad.responseInfo else { return nil }

    let firedProbeFloors: [Int] = resp.adNetworkInfoArray.compactMap { info in
        guard let label = info.adSourceInstanceName else { return nil }
        let range = NSRange(location: 0, length: label.utf16.count)
        guard let match = probeRegex.firstMatch(in: label, range: range),
              match.numberOfRanges > 1,
              let r = Range(match.range(at: 1), in: label)
        else { return nil }
        return Int(label[r])
    }.sorted()

    // 最低已触发探针 = AdMob 价格的最紧上界（瀑布流按 eCPM 降序遍历）。
    // 当没有探针触发时为 nil — 胜出方位于所有探针之上；没有可用上界。
    let lowestFired = firedProbeFloors.min()
    let proxyPrice = lowestFired

    return FloorProxyResult(
        firedProbeFloors: firedProbeFloors,
        lowestFired: lowestFired,

```

```

        proxyPrice: proxyPrice,
        winnerSourceName: resp.loadedAdNetworkResponseInfo?.adSourceName,
        winnerInstanceName: resp.loadedAdNetworkResponseInfo?.adSourceInstanceName,
        mgName: resp.extrasDictionary["mediation_group_name"] as? String
    )
}

// 用法
InterstitialAd.load(with: "<your-ad-unit-id>", request: Request()) { ad, error in
    guard let ad else {
        NSLog("[FloorProxy] load failed: %@", error?.localizedDescription ?? "unknown")
        return
    }
    if let result = computeFloorProxy(ad: ad) {
        NSLog("[FloorProxy] %@", "\(result)")
        // 生产环境：若 result.proxyPrice == nil，则默认由 AdMob 展示。
        // 否则，将 result.proxyPrice 与 CloudX 的并行出价进行比较。
    }
}

```

关于应读取哪个 `ResponseInfo` 字段。 触发后无填充的探针不会出现在 `loadedAdapterResponseInfo` 中（该字段仅描述最终展示广告的那个适配器）。它们的标签会出现在 `adapterResponses[*]`（Android）/ `adNetworkInfoArray`（iOS）中——这是所有适配器尝试的完整列表。读取错误的字段会导致代理返回 `null`；上述代码片段已正确遍历该字段。

使用Parameter字段的替代读取方法

相同的数据来源（`adapterResponses` / `adNetworkInfoArray`），不同的提取方式。如果你在每个探针Line Item（步骤1）的**Parameter**字段中填入类似 `{"floor": 12}` 的JSON对象，AdMob会在每个no-fill条目的 `credentials["parameter"]` 字段中暴露出来——因此你的读取逻辑可以直接获取并解析该字段为整数，而不用再用正则表达式去解析Label。对于字段为空的条目会返回 `null`，所以你可以同时混用仅Label和编码在Parameter中的Line Item 便于逐步迁移。

选择适合你实现的方式：Label方式每个Line Item少填一个管理后台字段；Parameter方式避免了客户端正则解析和Label重命名影响。两种方式都能实现相同的“最低触发探针”计算。

步骤 4 — 日志记录模板

将下面的代码加入您的应用，并在 `onAdLoaded` / `load` 完成回调中、计算代理之前调用。它会记录三个相关字段：胜出广告来源、所有适配器尝试（以便您查看哪些探针被触发）、以及 MG 名称。如果您需要更完整的转储用于通用 SDK 调试，AdMob 文档涵盖了 `ResponseInfo` 的所有字段。

Android (Kotlin) Example

```
fun dumpResponseInfo(ad: InterstitialAd, tag: String = "Floor
ProxyDump") {
    val resp = ad.responseInfo
    val mg = resp.responseExtras.getString("mediation_group_name") ?: "<unset>"
    android.util.Log.i(tag, "MG=$mg winner=${resp.loadedAdapterResponseInfo?.adSourceName} " +
        "(${resp.loadedAdapterResponseInfo?.adSourceInstanceName})")
    resp.adapterResponses.forEachIndexed { i, ar ->
        android.util.Log.i(tag, "  [$i] ${ar.adSourceName} /
        ${ar.adSourceInstanceName} " +
            "latency=${ar.latencyMillis}ms error=${ar.adError?.code} ?: "ok")
    }
}
```

iOS (Swift) Example

```
func dumpResponseInfo(_ ad: InterstitialAd, tag: String = "FloorProxyDump") {
    guard let resp = ad.responseInfo else { return }
    let mg = (resp.extrasDictionary["mediation_group_name"] as? String) ?? "<unset>"
    let loaded = resp.loadedAdNetworkResponseInfo
    NSLog("[%@] MG=%@ winner=%@ (%@)", tag, mg,
          loaded?.adSourceName ?? "nil", loaded?.adSourceInstanceName ?? "nil")
    for (i, info) in resp.adNetworkInfoArray.enumerated() {
        NSLog("[%@] [%d] %@ / %@ latency=%fs error=%@", tag, i,
              info.adSourceName ?? "nil", info.adSourceInstanceName ?? "nil",
              info.latency, info.error?.localizedDescription ?? "ok")
    }
}
```

部署注意事项

以下是您在线上过程中需要在集成层面做出的几项决策。在这些决策落地之后，您的 CloudX 客户经理（AM）会处理外围细节（探针层级策略、拍卖配置、效果度量）。

延迟（Latency）

探针机制本身不会增加广告加载时延（探针在 AdMob 的已有 waterfall 中触发；读取逻辑在 `onAdLoaded` 中执行）。下面关于时延的说明，主要针对 CloudX × AdMob 并行竞价集成——这是任何 CloudX 部署中都要考虑的问题。

并行竞价会在渲染前等待 CloudX 与 AdMob 两端的响应。建议的广告加载超时（ad-load timeout）：

- 美国流量：≥ 1,500 ms。
- 非美国流量：≥ 2,000 ms。

对于插屏广告（interstitial）和激励视频（rewarded video），并行模式效果显著——广告会在用户到达广告时机之前预加载，因此延迟影响很小。对于横幅（banner）和 MREC（中等矩形广告），请先评估 CloudX 的填充率（fill rate）/ 匹配率（match rate）；低填充 + 并行 = 由延迟驱动的损失。当填充率较低时，请考虑串行模式（先 CloudX，AdMob 兜底）。

部署后请监控广告加载延迟。 跟踪 AdMob 报告的延迟随时间的变化；启用 CloudX 后若延迟持续上升，则说明超时配置有误，会损失展示。如果您将本集成作为 A/B 测试运行（CloudX 实验组 vs. 对照广告单元），跨变体比较延迟是最干净的信号——您的 CloudX AM 可以协助搭建。

各广告形式的具体指引

> 如需根据自身业务情况微调上述建议，建议与 CloudX 客户经理联系。此外，不同广告场景及地区网络环境下的最优加载策略可能不同，持续监控并迭代调整配置，可获得最佳收益与体验。

集成清单

请在执行过程中逐项核对：

- ☐ 已创建广告单元（`arch4_test_interstitial` 或您的等效单元）（步骤 1）。
- ☐ 一个 MG，定向到该广告单元，并已设置竞价底价 `F`（步骤 1）。
- ☐ 已在 Bidding 区域添加真实竞价方（步骤 1）。
- ☐ 已在 Waterfall 区域添加自定义事件探针广告位，所有 eCPM 均高于 `F`（步骤 1）。
- ☐ 顶层探针已设置为远高于您预期的最高 AdMob 填充价——包括 AdMob Network 竞价模式的上行空间（保证每次填充时 `proxyPrice != null`）（步骤 1）。
- ☐ 探针密度已根据您的历史上 AdMob 填充集中的价位带进行调优（步骤 1）。
- ☐ AdMob Network 位于 Bidding 区域（默认模式）（步骤 1）。
- ☐ 已计算 `proxyPrice = lowestFired`（步骤 3）。
- ☐ 已配置 null 处理默认行为：当 `proxyPrice == null` 时由 AdMob 展示（不渲染 CloudX）；并对 null 比例进行监控，若超过约 1% 则提高顶层探针（步骤 3）。
- ☐ 已实现客户端读取器；遍历 `adapterResponses[*]`（而非 `loadedAdapterResponseInfo`）寻找 `Probe_F\` 标签（步骤 3）。

- ☐ 测试 Probe_F 的所有数值都可以正常读取，包括浮点数 0.5、15.5。
- ☐ 集成期间已埋设日志模板； `dumpResponseInfo` 在每次填充时执行（步骤 4）。
- ☐ 已配置广告加载超时：美国 $\geq 1,500\text{ ms}$ / 非美国 $\geq 2,000\text{ ms}$ （部署 > 延迟）。
- ☐ 部署后已建立延迟监控（部署 > 延迟）。
- ☐ 已就探针层级策略、拍卖配置和收益度量与 CloudX AM 对接。

译注：术语对照（IAB / 中国 RTB 广告语境）

英文	中文
Real-Time Bidding (RTB)	实时竞价 (RTB)
Parallel bidding	并行竞价
Mediation	聚合 (中介)
Mediation group (MG)	聚合组
Waterfall	瀑布流
Bidder / Bidding partner	竞价方 / 竞价合作伙伴
Bidder floor / Floor eCPM	竞价底价 / eCPM 底价
Manual eCPM	手动 eCPM
Custom event	自定义事件
Adapter	适配器
Ad unit	广告单元
Ad source	广告来源
Impression-Level Revenue Data (ILRD)	单次展示收益数据 (ILRD)
Fill / No-fill	填充 / 无填充
Fill rate	填充率
Match rate	匹配率
Latency	延迟
Interstitial	插屏广告
Rewarded video	激励视频
Banner	横幅广告

英文	中文
MREC	中等矩形广告 (MREC)
Demand source	需求来源
Invalid traffic	无效流量
User Value Segmentation (UVS)	用户价值分层 (UVS)